

INTERNSHIP REPORT

CYBERSECURITY DOMAIN

Securing the Digital Future

Submitted By: Adil Rehan

Email: rehanrecon1@gmail.com

Organization: Vault-Tec Security

Duration: 02 Months

Internship: 2nd Month

Date of Submission: 31st March 2026

TABLE OF CONTENTS

1. OWASP Top 10 2025: IAAA Failures
2. OWASP Top 10 2025: Application Design Flaws
3. OWASP Top 10 2025: Insecure Data Handling

OWASP Top 10 IAAA Failures

Introduction

The screenshot shows a web browser window with the URL `tryhackme.com/room/owasptopten2025one` in the address bar. The page title is "OWASP Top 10 2025: IAAA Failures". Below the title, it says "Learn about A01, A07, and A09 in how they related to failures in the applied IAAA model." and shows a timer for 30 minutes and 30,321 participants. There are buttons for "Start AttackBox", "Save Room", "202 Recommend", and "Options". A notification banner at the top states: "Our Asia Pacific VM region is now fully available. You can now use all features in this region." Below this, a task header "Task 1 Introduction" is visible. The main content area features the OWASP logo and a paragraph: "This room breaks down 3 of the OWASP Top 10 2025 categories. In this room, you will learn about the categories that are related to failures in how Identity, Authentication, Authorisation, and Accountability (IAAA) is implemented in the application. You will put the theory into practice by completing supporting challenges. The following categories are covered in this room: 1. A01: Broken Access Control 2. A07: Authentication Failures 3. A09: Logging & Alerting Failures. The room has been designed for beginners and assumes no previous security knowledge." Below this, it says "Answer the questions below" and "I am ready to learn about IAAA failures!". At the bottom, there is a "No answer needed" button and a green "Correct Answer" button.

tryhackme.com/room/owasptopten2025one

OWASP Top 10 2025: IAAA Failures

Learn about A01, A07, and A09 in how they related to failures in the applied IAAA model.

30 min 30,321

Start AttackBox Save Room 202 Recommend Options

Room progress (12%)

Our Asia Pacific VM region is now fully available. You can now use all features in this region. [More info](#)

Task 1 Introduction

This room breaks down 3 of the OWASP Top 10 2025 categories. In this room, you will learn about the categories that are related to failures in how Identity, Authentication, Authorisation, and Accountability (IAAA) is implemented in the application. You will put the theory into practice by completing supporting challenges. The following categories are covered in this room:

1. A01: Broken Access Control
2. A07: Authentication Failures
3. A09: Logging & Alerting Failures

The room has been designed for beginners and assumes no previous security knowledge.

Answer the questions below

I am ready to learn about IAAA failures!

No answer needed [Correct Answer](#)

What is IAAA



OWASP Top 10 IAAA Failures

A01: Broken Access Control

The screenshot displays a TryHackMe room interface for the task 'A01: Broken Access Control'. The room progress is at 44%.

Task 3: A01: Broken Access Control

Broken Access Control happens when the server doesn't properly enforce **who can access what** on every request. A common occurrence of this is **IDOR** (Insecure Direct Object Reference): if changing an ID (like `?id=7` to `?id=6`) lets you see or edit someone else's data, access control is broken.

In practice this shows up as horizontal privilege escalation (same role, other user's stuff) or vertical privilege escalation (jumping to admin-only actions) because the application trusts the client too much.

Start the static site attached to this task and play with the `accountID` value in the URL. So if you can identify which user has more than \$1 million in their account!

If you want a deeper dive or more variations on the theme (encoded IDs, hashed IDs, etc.), work through these rooms afterwards:

- Broken Access Control
- Insecure Direct Object References

Answer the questions below

If you don't get access to more roles but can view the data of another users, what type of privilege escalation is this?

Horizontal ☒ ✓ Correct Answer

What is the note you found when viewing the user's account who had more than \$1 million?

THM{Found.the.Millionare!} ☒ ✓ Correct Answer

Task 4: A07: Authentication Failures

Task 5: A09: Logging & Alerting Failures

Task 6: Conclusion

Vulnerability Detected

This application contains an **Insecure Direct Object Reference (IDOR)** vulnerability. By simply changing the ID parameter in the URL, you can access other users' account information without proper authorization checks.

Try changing the `id` value in the URL bar above to view different accounts (1-10).

How to Fix This Vulnerability

- Implement proper authentication and authorization checks
- Verify that the authenticated user has permission to access the requested resource
- Use indirect references or encrypted tokens instead of predictable IDs
- Log and monitor access attempts to sensitive resources

OWASP 2025: IAAA.1

OWASP Top 10 IAAA Failures

A07: Authentication Failures

Task 4 **A07: Authentication Failures** Room progress (50%)

Authentication Failures happen when an application can't reliably verify or bind a user's identity. Common issues include:

- username enumeration
- weak/guessable passwords (no logout/rate limits)
- logic flaws in the login/registration flow
- insecure session or cookie handling

If any of these are present, an attacker can often log in as someone else or bind a session to the wrong account.

Let's try to break into the `admin` user's account. We know that their username is `admin`, so let's try to fool the application by registering a user with the name of `admin`. Start the static site attached to this task, register your account and log into the admin user's account to get your next flag!

If you want more depth or broader techniques (e.g., brute force, session handling, cookies/JWT/OAuth, and MFA specifics), work through these after this room:

- [Authentication Bypass Room](#)
- [Multi-Factor Authentication](#)
- [Authentication Module](#)

Answer the questions below

What is the flag on the `admin` user's dashboard?

`THM{Account.confusion.FTW!}` **Correct Answer**

Task 5 **A09: Logging & Alerting Failures**

Task 6 **Conclusion**

Account Dashboard Logout

Welcome back, admin

Account Balance

\$1,000,000.00

Account Description:

`THM{Account.confusion.FTW!}`

Account Information

Username	admin
Account ID	account-...
Member Since	2/3/2026

OWASP 2025: IAAA 1 OWASP 2025: IAAA 2

OWASP Top 10 IAAA Failures

A09: Logging & Alerting Failures

Room progress (88%)

Task 5 A09: Logging & Alerting Failures

When applications don't record or alert on security-relevant events, defenders can't detect or investigate attacks. Good logging underpins **accountability** (being able to prove who did what, when, and from where). In practice, failures look like missing authentication events, vague error logs, no alerting on brute-force or privilege changes, short retention, or logs stored where attackers can tamper with them.

Let's take a look at what is required to perform an investigation of an application under attack. Start the static site attached to this task, perform your investigation, and answer the questions below. Then, think about how hard it would be to understand what happened during this attack if key pieces of this log information as missing.

If you want a deeper dive into logging for accountability, take a look at [this room](#).

Answer the questions below

It looks like an attacker tried to perform a brute-force attack, what is the IP of the attacker?

Correct Answer

Looks like they were able to gain access to an account! What is the username associated with that account?

Correct Answer

What action did the attacker try to do with the account? List the endpoint the accessed.

Correct Answer

Task 6 Conclusion

08:35:14

2025-01-15 08:36:37	200 OK	208.88.152.45	GET	/cards
2025-01-15 08:38:00	401 Unauthorized	151.88.31.167	POST	/login?user=sarah&pass=test123
2025-01-15 08:39:23	200 OK	172.217.22.67	GET	/loans
2025-01-15 08:40:46	200 OK	23.211.188.78	GET	/support
2025-01-15 08:41:12	200 OK	54.239.28.85	POST	/deposit
2025-01-15 08:41:58	302 Redirect	142.250.185.99	GET	/mobile
2025-01-15 08:42:09	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=admin123
2025-01-15 08:42:13	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=password
2025-01-15 08:42:17	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=12345
2025-01-15 08:42:21	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=letmein
2025-01-15 08:42:25	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=qwerty
2025-01-15 08:42:29	200 OK	203.0.113.45	POST	/login?user=admin&pass=password123
2025-01-15 08:42:31	200 OK	203.0.113.45	GET	/supersecretadminstuff
2025-01-15 08:43:22	200 OK	13.107.42.14	GET	/statements

OWASP 2025: IAAA.1 OWASP 2025: IAAA.2 OWASP 2025: IAAA.3

Conclusion

✓ Correct Answer

OWASP Top 10 Application Design Flaws

Introduction

The screenshot shows a web browser window with the URL `tryhackme.com/room/owasptopten2025two` highlighted in a red box. The page is titled "Room progress (16%)" and features a "Start AttackBox" button. Below this, there is a "Target machine" section with a status of "Off" and a "Start Machine" button. The main content area displays the OWASP logo and a list of categories: 1. AS02: Security Misconfigurations, 2. AS03: Software Supply Chain Failures, 3. AS04: Cryptographic Failures, and 4. AS06: Insecure Design. A "Deploy Practical" section follows, with instructions to click the "Start Machine" button. At the bottom, there is a "Correct Answer" button highlighted in a red box.

Room progress (16%)

Status: Off Start AttackBox

Target machine

Status: Off Start Machine

OWASP

This room breaks each 4 of the OWASP Top 10 2025 categories. In this room, you will learn about the categories that are related to failures in architecture and system design. You will put the theory into practice by completing supporting challenges. The following categories are covered in this room:

1. AS02: Security Misconfigurations
2. AS03: Software Supply Chain Failures
3. AS04: Cryptographic Failures
4. AS06: Insecure Design

Deploy Practical

Before we begin, please deploy the practical VM by clicking the green "Start Machine" button at the start of this task. Please note that you will need to use either the TryHackMe AttackBox or your own hacking machine connected to the TryHackMe VPN to access each practical.

Answer the questions below

I am ready to learn about design flaw vulnerabilities!

No answer needed

Correct Answer

OWASP Top 10 Application Design Flaws

AS02: Security Misconfigurations

The screenshot displays a TryHackMe room interface. The browser address bar shows `tryhackme.com/room/owasptopten2025two`. The room progress is 33%. The active machines information shows IP addresses `10.49.174.4` and `10.49.179.130`.

How To Prevent It

- Harden default configurations and remove unused features or services
- Enforce strong authentication and least privilege across all systems
- Limit network exposure and segment sensitive resources
- Keep software, frameworks, and containers up to date with patches
- Hide stack traces and system information from error messages
- Audit cloud configurations and permissions regularly
- Secure AI endpoints and automation services with proper access controls and monitoring
- Integrate configuration reviews and automated security checks into your deployment pipeline

Challenge

Navigate to `10.49.179.130:5002`. It appears that the developers left too many traces in their User Management APIs.

Answer the questions below

What's the flag?

`THM{V3RB0S3_3RR0R_L34K}` ✓ Correct Answer

The terminal window on the right shows a command prompt on `root@ip-10-49-174-4:~`. The output of a command is a JSON object:

```
{
  "debug_info": {
    "flag": "THM{V3RB0S3_3RR0R_L34K}"
  },
  "error": "Invalid user ID format: api. Flag: THM{V3RB0S3_3RR0R_L34K}",
  "traceback": "Traceback (most recent call last):\n File \"/ap\n/app.py", line 21, in get_user\n raise ValueError(f\"Invali\n user ID format: {user_id}. Flag: {FLAG}\")\nValueError: Invali\n user ID format: api. Flag: THM{V3RB0S3_3RR0R_L34K}\n"
```

OWASP Top 10 Application Design Flaws

AS03: Software Supply Chain Failures

Room progress (50%)

Active machines information

IP Address: 10.49.174.4

IP Address: 10.49.179.130

How To Protect The Supply Chain

- Verify all third-party components, libraries, and AI models before use
- Monitor and patch dependencies regularly
- Sign, verify, and audit software updates and packages
- Lock down CI/CD pipelines and build processes to prevent tampering
- Track provenance and licensing for all dependencies
- Implement runtime monitoring for unusual behaviour from dependencies or AI components
- Integrate supply chain threat modelling into the SDLC, including testing, deployment, and update workflows

Challenge

Navigate to 10.49.179.130:5003. The code is outdated and imports an old lib/vulnerable_utils.py component. Can you *debug* it?

Answer the questions below

What's the flag?

THM{SUPPLY_CHAIN_VULN3R4B1L1TY}

✓ Correct Answer

THM AttackBox 5h 44min 56s

Burp Suite Community Edition v2024.9.5 - Temporary Project

Target: http://10.49.179.130:5003

Request

```
POST /api/process HTTP/1.1
Host: 10.49.179.130:5003
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-GB,en;q=0.9
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Type: application/json
Content-Length: 18

{"data": "debug"}
```

Response

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.11.14
Date: Mon, 02 Mar 2026 08:06:57 GMT
Content-Type: application/json
Content-Length: 157
Connection: close

{"admin token": "admin token 12345",
"flag": "THM{SUPPLY_CHAIN_VULN3R4B1L1TY}",
"internal_secret": "internal_secret_key_2024",
"version": "1.2.3"}
```

OWASP Top 10 Application Design Flaws

AS04: Cryptographic Failures

The screenshot displays a THM AttackBox interface. On the left, a sidebar shows 'Active machines information' with IP addresses 10.49.149.50 and 10.49.151.159. The main content area shows a challenge titled '1.3 with valid certificates' with a list of bullet points. Below the challenge, a 'Challenge' section asks the user to navigate to 10.49.151.159:5004 and find a key to decrypt a file. The answer field contains the flag 'THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}' and a green 'Correct Answer' button. On the right, a web browser window shows a 'Secure Document Viewer' page. The page contains a message about a confidential document and a note that the decryption feature is unavailable. A red box highlights the 'Encrypted Document' section, which contains a long base64-encoded string.

tryhackme.com/room/owasptopten2025two

Room progress (16%)

Active machines information

IP Address: 10.49.149.50 IP Address: 10.49.151.159

1.3 with valid certificates

- Use secure key management services like Azure Key Vault, AWS KMS, or HashiCorp Vault
- Rotate secrets and keys regularly, following defined crypto periods
- Document and enforce policies and standard operating procedures for key lifecycle management
- Maintain a complete inventory of certificates, keys, and their owners
- Ensure AI models and automation agents never expose unencrypted secrets or sensitive data
- The web application in this room contains a weakness of this type for you to explore.

Challenge

Navigate to 10.49.151.159:5004. Can you find the key to decrypt the file?

Answer the questions below

What's the flag?

THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y} ✓ Correct Answer

Secure Document Viewer

http://10.49.151.159:5004

CyberChef Reverse Shell Generator

Secure Document Viewer

This confidential document is encrypted for security. Only authorized personnel can access the decryption key.

Note: The decryption feature is currently unavailable. Contact your administrator for access.

Encrypted Document:

Nzd4ZH2GgU1UlpILZRv0jeIXp1WtCErW+j/w/LnKbmug31opX8BWy+pwK92rkhjwdF94mgHfLtF26X683pe2fhHXzIGnnvVruh7683KwvzZ6+QKybFWaedAETknYkhe

THM AttackBox 6h 8min 38s

OWASP Top 10 Application Design Flaws

AS06: Insecure Design

tryhackme.com/room/owasptopen2025two

Room completed (100%)

?

Add 1 hour

Terminate

Separate system prompts from user content.

- Keep sensitive data out of prompts unless absolutely needed and protect it with strict controls.
- Require human review for high-risk AI actions.
- Log model provenance, monitor behaviour, and apply differential privacy for sensitive data.
- Include AI-specific threat modelling for prompt attacks, inference risks, agent misuse, and supply chain compromise throughout the design process.
- Build threat modelling into every stage of development, not just at the start.
- Define clear security requirements for each feature before implementation.
- Apply the principle of least privilege across users, APIs, and services.
- Ensure proper authentication, authorisation, and session management across the system.
- Keep dependencies, third-party components, and supply chain sources verified and up to date.
- Continuously monitor and test the system for logic flaws, abuse paths, and emergent risks as new features or AI components are added.

Challenge

Navigate to `10.49.174.104:5005`. Have they assumed that only mobile devices can access it?

Answer the questions below

What's the flag?

THM{1NS3CUR3_D3SIGN_4SSUMPT10N}

✓ Correct Answer

OWASP Top 10 Application Design Flaws

Conclusion

The screenshot shows a web browser window with the URL `tryhackme.com/room/owasptopen2025two` highlighted in a red box. The browser's address bar also shows a 110% zoom level and various extension icons. Below the browser window, a blue header bar indicates 'Room progress (83%)'. The main content area is divided into two sections: 'Active machines information' and 'Task 6 Conclusion'. The 'Task 6 Conclusion' section is expanded, showing the title 'Conclusion' and a paragraph of text. The text discusses security design failures across various categories (AS02 Security Misconfigurations, AS03 Software Supply Chain Failures, AS04 Cryptographic Failures, and AS06 Insecure Design) and states that they all stem from weak foundations. It emphasizes the importance of clear security requirements, realistic threat assumptions, controlled configurations, vetted dependencies, and sound cryptographic choices. The text also advises treating defaults with suspicion, treating every dependency as a potential risk, and keeping design simple enough to reason about. A link is provided to continue the journey with Room 3 in this module: <https://tryhackme.com/jr/owasptop102025insecuradatahandling>. Below the text, there is a green button labeled 'Correct Answer' and a grey button labeled 'No answer needed'.

tryhackme.com/room/owasptopen2025two

Room progress (83%)

Active machines information

IP Address: 10.49.174.104

Task 6 Conclusion

Conclusion

Security design failures across AS02 Security Misconfigurations, AS03 Software Supply Chain Failures, AS04 Cryptographic Failures, and AS06 Insecure Design all come from the same root cause: weak foundations. You cannot add security at the end and expect it to work. Strong systems start with clear security requirements, realistic threat assumptions, controlled configurations, vetted dependencies, and sound cryptographic choices.

Treat defaults with suspicion, treat every dependency as a potential risk, and keep design simple enough to reason about. Get the design right early, and you avoid a long future of preventable incidents.

Continue the journey with **Room 3 in this module: Application Design Flaws**: <https://tryhackme.com/jr/owasptop102025insecuradatahandling>

Answer the questions below

I'm ready for the next room!

No answer needed

✓ Correct Answer

OWASP Top 10 Insecure Data Handling

Introduction

The screenshot shows a web browser window with the URL `tryhackme.com/room/owasptopten2025three` in the address bar. The page is titled "Task 1: Introduction" and contains the following content:

This room will introduce you to 3 elements of the OWASP Top 10 list (2025). In this room, you will learn about the elements relating to application behaviour and user input. We will cover these vulnerabilities briefly, how to prevent them, and finally, you will practice exploiting these vulnerabilities:

- A04: Cryptographic Failures
- A05: Injection
- A08: Software or Data Integrity Failures

Deploy Practical

Before we begin, please deploy the practical VM by clicking the green "Start Machine" button below. Please note that you will need to use either the TryHackMe AttackBox or your own hacking machine connected to the TryHackMe VPN to access each practical.

Set up your virtual environment

To successfully complete this room, you'll need to set up your virtual environment. This involves starting both your AttackBox (if you're not using your VPN) and Target Machines, ensuring you're equipped with the necessary tools and access to tackle the challenges ahead.

The "Set up your virtual environment" section contains two cards:

- Attacker machine**: Status: Off. A "Start AttackBox" button is visible.
- Target machine**: Status: Off. A green "Start Machine" button is visible.

If you're ready, let's begin the showcase.

Answer the questions below

I'm ready!

No answer needed

Correct Answer

OWASP Top 10 Insecure Data Handling

A04: Cryptographic Failures

The screenshot displays a TryHackMe room interface. On the left, the 'Practical' section for task A04: Cryptographic Failures is visible. It provides instructions to decrypt notes on a web application at <http://10.48.128.144:8001>. Below the instructions, a text input field contains the placeholder `THM{WEAK_CRYPTO_FLAG}`, which is highlighted with a red box. To the right of the input field is a green button labeled 'Correct Answer' and an orange button with a question mark. The 'Recommended TryHackMe Content' section lists the 'Cryptographic Failures Module'.

On the right side of the interface, a terminal window titled 'Crypto Lab: Weak XOR Cipher' is open. The terminal shows the URL <http://10.48.128.144:8001> in the address bar, also highlighted with a red box. The terminal output shows a 'Decrypted with key "KEY1"' message, followed by a 'Password reset link: https://internal.thm/reset?token=abc123'. Below this, a section titled 'Confidential #3' shows 'Encrypted (base64)' data. The decrypted output is 'Decrypted with key "KEY1"' followed by 'SECRET: THM{WEAK_CRYPTO_FLAG}' and a warning 'Do not share this with unauthorized personnel.' The 'SECRET' text is highlighted with a red box. The terminal window also shows a 'THM AttackBox' logo and a timer at '20min 52s'.

OWASP Top 10 Insecure Data Handling

A05: Injection

The image shows a TryHackMe room interface on the left and an SSTI Playground tool on the right.

TryHackMe Room Interface:

- Room progress: 75%
- Active machines information: IP Address: 10.48.168.170, IP Address: 10.48.128.144
- You can access this portion of the practical on <http://10.48.128.144:8000>.
- Recommended TryHackMe Content**
- If you'd like to explore this type of attack in much further depth, we highly recommend the following TryHackMe content:
 - Injection Attacks Module
 - Command Injection
- Answer the questions below
- Perform an SSTI attack on the practical. You need to read the contents of flag.txt that is located within the same directory as the web application.
- Input field: THM{SSTI_FLAG_OBTAINED} (Correct Answer)
- Task 4: A08: Software or Data Integrity Failures
- How likely are you to recommend this room to others? (1-10 scale)
- Submit now

SSTI Playground:

- Injection Showcase
- URL: <http://10.48.128.144:8000>
- Tools: CyberChef, Reverse Shell Generator
- SSTI Playground**
- This lab intentionally renders raw user input inside a Jinja2 template using `render_template_string`. Enter snippets below to observe how template expressions execute on the server.
- PAYLOAD:

```
{{ request.application.__globals__.__builtins__.open('flag.txt').read() }}
```
- Render payload
- Rendered Output: THM{SSTI_FLAG_OBTAINED} (Untrusted • Dangerous)
- THM AttackBox
- 8min 38s

OWASP Top 10 Insecure Data Handling

A08: Software or Data Integrity Failures

The image shows a TryHackMe room interface on the left and a terminal window on the right. The TryHackMe room is titled "Room completed [100%]" and shows "Active machines information" with IP addresses 10.48.168.170 and 10.48.128.144. The main content area discusses preventing failures by establishing trust boundaries and includes a "Practical" section about a deserialization attack. Below this is a "Recommended TryHackMe Content" section with links to "Insecure Deserialisation" and "Supply Chain Attack: Lottie". A question asks for the contents of flag.txt, and the answer "THM{INSECURE_DESERIALIZATION}" is entered in a text box, marked as correct. The terminal window on the right shows a "Deserialize Object" application with a "PICKLE DATA (BASE64)" input field containing the same base64-encoded string, a "Deserialize" button, and a "Deserialized Output" field showing the decoded flag.

tryhackme.com/room/owasptopten2025three

Room completed [100%]

Active machines information

IP Address: 10.48.168.170 IP Address: 10.48.128.144

Preventing these failures begins with establishing trust boundaries. Applications should never assume that code, updates, or key pieces of data are legitimate and automatically trusted; their integrity must be verified. This involves using methods such as cryptographic checks (like checksums) for update packages and ensuring that only trusted sources can modify critical artefacts.

Additionally, for applications, integrity and trust boundaries should also be within build processes such as CI/CD.

Practical

This practical will demonstrate a deserialization attack in Python. You can access this practical on <http://10.48.128.144:8002>.

Follow the steps in the practical to generate and provide some malicious input to the web application.

Recommended TryHackMe Content

If you'd like to explore this type of attack in much further depth, we highly recommend the following TryHackMe content:

- Insecure Deserialisation
- Supply Chain Attack: Lottie

Answer the questions below

Use Python to pickle a malicious, serialised payload that reads the contents of flag.txt and submits it to the application.

What are the contents of flag.txt?

THM{INSECURE_DESERIALIZATION} ✓ Correct Answer

How likely are you to recommend this room to others?

Integrity Lab: Insecure D

http://10.48.128.144:8002

CyberChef Reverse Shell Generator

Accessers can create malicious pickle payloads to execute arbitrary code and access sensitive files.

Deserialize Object

Enter base64-encoded pickle data below. The application will deserialize it without any integrity verification.

PICKLE DATA (BASE64)

gASVWmAAAAAACMGJ1aWx0aW5lIiwEZXB3STLIwB38lbgZnZmxhZy50eHhKSS5yZWFKKCMhZRS1C4=

Deserialize

Deserialized Output

THM{INSECURE_DESERIALIZATION}

THM AttackBox 3min 6s